La cafetería de la ETSII, situada en el corazón de la escuela, es desde hace años el punto de encuentro del alumnado y del profesorado a la hora de desayunar, tomar algún bocado entre clases o reponer fuerzas para los exámenes. Hasta ahora, a Julio, su gerente, le bastaba con saber, a final de mes, cuánto dinero había entrado en caja, pero ahora quiere ir más allá: quiere entender a qué horas se llena más el local, qué mesas dejan mejor propina, qué categorías de producto tiran del negocio en cada franja y muchas otras cuestiones similares. Para ello, ha empezado a registrar, pedido a pedido, todo lo que ocurre en su caja, y pide ayuda a los alumnos de FP para analizar esa información.

Se dispone de un archivo en formato CSV codificado en UTF-8 con información detallada de cada producto de la carta. El objetivo de este examen es implementar una serie de funciones en Python que permitan cargar, filtrar y analizar estos datos utilizando estructuras de datos avanzadas. Cada línea del archivo contiene los siguientes datos:

- **Id del pedido**: Identificador numérico único de cada pedido.
- **Fecha y hora del pedido**: Día y hora en que se registró el pedido en caja, con formato YYYY-MM-DD HH:MM.
- **Mesa:** Identificador de la mesa donde se sirvió el pedido, o el valor "Barra" si se atendió en el mostrador.
- **Método Pago**: Forma de pago utilizada (ej: "Efectivo", "Tarjeta", "Bizum").
- **Propina**: Importe en euros de la propina dejada en ese pedido (puede ser 0.0).
- **Líneas**: Relación de las líneas de los productos solicitados en el pedido. Cada línea de pedido se define por cuatro campos unidos por el carácter "-" (nombre del producto, categoría, cantidad y precio unitario en euros).Las distintas líneas se separan entre sí mediante asteriscos "*".

Ejemplo de registro:

```
145; 2025-09-10 09:15; Mesa_3; Tarjeta; 0.50; Cafe_Con_Leche-Bebida-2-1.50*Napolitana_Chocolate-Dulce-1-1.80
```

En este caso:

- El Id del pedido es 145.
- Se registró el 10 de septiembre de 2025 a las 09:15.
- Fue en la Mesa_3, se usó la tarjeta como medio de pago y se dejó una propina de 0.50 €.
- Contiene dos líneas de pedido:
  - 2 unidades de "Cafe_Con_Leche" (categoría Bebida) a 1.50 € cada una.
  - 1 unidad de "Napolitana_Chocolate" (categoría "Dulce") a 1.80 €.

Estructura del proyecto

El proyecto a entregar debe estar escrito en Python, y contener la siguiente estructura de archivos, estando en la carpeta ***src*** los ficheros fuente del proyecto; en la carpeta ***data*** el fichero de datos que debe analizar; y en la carpeta ***doc***, el enunciado de tareas de análisis a realizar, así como el fichero de ***identificación.md*** **que debe rellenar con sus datos**:

- /src: contiene cafeteria.py y cafeteria_test.py.
- /data: contiene pedidos.csv.
- /doc: contiene identificacion.md y el enunciado en PDF.

Tipo NamedTuple

Para modelar de manera consistente los datos proporcionados, se deben usar las siguientes definiciones de NamedTuple:

```
from typing import NamedTuple
from datetime import datetime

LineaPedido = NamedTuple("LineaPedido", [
    ("producto", str),
    ("categoria", str),
    ("cantidad", int),
    ("precio_unitario", float)
])

Pedido = NamedTuple("Pedido", [
    ("id_pedido", int),
    ("fecha_hora", datetime),
    ("mesa", str),
    ("metodo_pago", str),
    ("propina", float),
    ("lineas", list[LineaPedido])
])
```

Ejercicios

Las funciones que resuelvan los ejercicios a realizar se deberán implementar en el módulo Python ***cafeteria.py***. Por otro lado, en el módulo ***cafeteria_test.py*** debe implementar lo necesario para probar la correcta ejecución de dichas funciones. Las puntuaciones están indicadas para cada ejercicio, y solo se podrá obtener la puntuación máxima si se implementa la función de test correspondiente, ésta tiene los parámetros adecuados, se puede ejecutar y el resultado coincide con el esperado.

Ejercicio 1 (1.5 puntos)

lee_pedidos: Lo primero que te solicita Julio es que seas capaz de leer el fichero de datos que te proporciona codificado en formato “utf-8”. Para ello pide implementar la función lee_pedidos que recibe la ruta de un fichero CSV como una cadena y devuelve una lista de tuplas de tipo Pedido con los datos leídos del fichero.

Se recuerda que los especificadores de formato que se necesitan para construir la cadena de formato que permite parsear la fecha y hora de los pedidos son para: el año “%Y”, el mes “%m”, el día “%d”, la hora “%H” y los minutos “%M”.

```
def lee_pedidos(
      ruta_fichero: str
) -> list[Pedido]
```

Salida esperada:

```
*********Ejercicio 1*********
TEST lee_pedidos
Número de registros leídos: 100

Los tres primeros registros son:
[Pedido(id_pedido=101, fecha_hora=datetime.datetime(2025, 9, 10, 8,15), mesa='Barra',
metodo_pago='Efectivo', propina=0.0, lineas=[LineaPedido(producto='Cafe_Con_Leche',
categoria='Bebida', cantidad=1, precio_unitario=1.5), LineaPedido(producto='Croissant',
categoria='Dulce', cantidad=1, precio_unitario=1.6)]),

Pedido(id_pedido=102, fecha_hora=datetime.datetime(2025, 9, 10, 8, 30), mesa='Mesa_1',
metodo_pago='Tarjeta', propina=0.5, lineas=[LineaPedido(producto='Cafe_Con_Leche',
categoria='Bebida', cantidad=2, precio_unitario=1.5),
LineaPedido(producto='Napolitana_Chocolate', categoria='Dulce', cantidad=2,
precio_unitario=1.8)]),

Pedido(id_pedido=103, fecha_hora=datetime.datetime(2025, 9, 10, 9, 0), mesa='Mesa_2',
metodo_pago='Bizum', propina=1.2, lineas=[LineaPedido(producto='Zumo_Naranja',
categoria='Bebida', cantidad=1, precio_unitario=2.5),
LineaPedido(producto='Tostada_Aceite', categoria='Salado', cantidad=1,
precio_unitario=1.8), LineaPedido(producto='Cafe_Solo', categoria='Bebida', cantidad=1,
precio_unitario=1.2)])]
```

Ejercicio 2 (1.75 puntos)

ingreso_total_por_franja: Julio quiere entender en qué franjas horarias se concentra el negocio, así que solicita implementar una función que reciba la lista de pedidos, una hora de inicio y una hora de fin (ambas como números enteros entre 0 y 23) y, de forma opcional, un método de pago (cuyo valor por defecto es None). La función debe devolver el ingreso total de los pedidos cuya hora de registro, sin tener en cuenta los minutos, esté comprendida entre la hora de inicio (incluida) y la hora de fin (excluida). Si se ha indicado un método de pago, solo deben contabilizarse los pedidos realizados con ese método de pago.

Nota: El ingreso de un pedido es la suma de los importes de todas sus líneas (*contando también la propina*), y el importe de una línea se calcula como el producto de su cantidad por su precio unitario.

```
def ingreso_total_por_franja(
                  pedidos: list[Pedido],
                  hora_inicio: int,
                  hora_fin: int,
                  metodo_pago: str|None = None
) -> float
```

Salida esperada:

```
*********Ejercicio 2*********
TEST ingreso_total_por_franja '8-12'
Ingreso total: 276.8 €
TEST ingreso_total_por_franja '12-18' con método de pago Tarjeta
Ingreso total: 246.7 €
TEST ingreso_total_por_franja '18-24' con método de pago Bizum
Ingreso total: 41.5 €
```

Ejercicio 3 (2 puntos)

analisis_propinas_ubicacion: Con el objetivo de ver cómo influyen las propinas, Julio solicita que implementes una función que reciba una lista de tuplas de tipo Pedido y un método de pago, y que devuelva la identificación de la mesa con mayor promedio de porcentaje de propina respecto al coste del pedido. El porcentaje de propina de cada pedido se calcula multiplicando por 100 el valor de la propina y dividiéndolo por el coste total del pedido**, excluida la propina**.

```
def analisis_propinas_ubicacion(
      pedidos: list[Pedido],
      metodo_pago: str
) -> tuple[str, float]
```

Salida esperada:

```
*********Ejercicio 3*********
TEST análisis propinas por ubicación para método de pago: Efectivo
Ubicación con el promedio de propina más alto: Mesa_5 (16.52%)

TEST análisis propinas por ubicación para método de pago: Tarjeta
Ubicación con el promedio de propina más alto: Mesa_5 (19.20%)

TEST análisis propinas por ubicación para método de pago: Bizum
Ubicación con el promedio de propina más alto: Mesa_2 (21.82%)
```

Ejercicio 4 (2,25 puntos)

producto_mas_asociado: Para diseñar ofertas, Julio necesita saber qué productos se compran juntos. Para ello, pide que implementes una función que reciba una lista de tuplas de tipo Pedido y el nombre de un producto, y que devuelva el nombre del *otro* producto que se ha pedido con mayor frecuencia en combinación con el producto de referencia. Ten en cuenta que un producto solo debe contarse una vez por pedido a efectos de la asociación, aunque se hayan pedido varias unidades del mismo (por ejemplo, dos cafés con leche).

```
def producto_mas_asociado(
      pedidos: list[Pedido],
      nombre_producto:str
) -> str
```

Salida esperada:

```
*********Ejercicio 4*********
TEST producto_mas_asociado para producto: Hamburguesa_Completa
Producto más asociado: Cerveza

TEST producto_mas_asociado para producto: Cafe_Con_Leche
Producto más asociado: Napolitana_Chocolate
```

Ejercicio 5 (2,5 puntos)

mesas_gourmet: Para identificar y dar un trato preferente a los clientes que realizan un mayor consumo en la sala, Julio quiere localizar las llamadas "Mesas Gourmet". Para ello, se pide que implementes una función que reciba una lista de tuplas de tipo Pedido y un entero que represente una hora, y devuelva una lista con los identificadores de las mesas cuyo gasto medio por pedido a esa hora concreta (**excluida la propina)** sea superior en más de un 40% al gasto medio global de la cafetería en esa misma hora. En ambos cálculos, el de cada mesa y el global, deben excluirse por completo los registros de la "Barra". Si la hora es None, se tendrán en cuenta todos los pedidos.

```
def mesas_gourmet(
      pedidos: list[Pedido],
      hora: int|None = None
) -> list[str]
```

Salida esperada:

```
*********Ejercicio 5*********
TEST mesas_gourmet para hora: None
Mesas gourmet: ['Mesa_5']

TEST mesas_gourmet para hora: 12
Mesas gourmet: []
```